

MOVR SYSTEM ARCHITECTURE & ENGINEERING ROADMAP

Comprehensive Review of the Current Codebase, Vulnerability Analysis, and Enterprise-Scale Strategies (Uber H3, Redis Geolocation, Dynamic Surge, Corridor Routing, and Streamed Telemetry)

Branch Evaluated: claude/nifty-newton-DhwTT	Target Stack: Flutter (Dart 3.5) + Django 5.1 / Channels	Date: August 2026
--	--	-------------------

1. Executive Overview & Platform Concept

Movr is a hybrid peer-to-peer mobility and logistics marketplace tailored for urban transportation ecosystems (specifically Nigeria). It connects daily commuters and commercial drivers (**'Movers'**) with passengers seeking rides (**'Ride Sharing'**) and individuals/businesses sending packages (**'Parcel Delivery'**) along identical transit corridors. The system incorporates digital wallet provisioning (Monnify virtual accounts / Paystack), KYC validation, dual verification handovers (QR/OTP), and real-time live location tracking via WebSockets.

Table 1.1: Core Stack Component Breakdown

Layer	Technology & Frameworks	Key Responsibilities & Design Patterns
Mobile Client	Flutter 3.27, Dart 3.5, Riverpod 2.5, Dio 5.8, Google Maps Flutter, Mobile Scanner, Rive	Cross-platform UI (88 presentation screens), Auth Route Guarding, JWT auto-refresh interceptors, TaskStateSync event bus, live polylines & animated Rive mover pins.
Backend API	Python 3.12, Django 5.1, Django REST Framework 3.15, SimpleJWT	RESTful endpoints under /api/, input sanitization filters, scoped rate throttling, Fernet encrypted PII columns (BVN/NIN), full KYC state machine.
Realtime Engine	Django Channels 4.1, Daphne 4.1 (ASGI), Redis 7 (channels-redis)	WebSocket tracking under ws/tracking/, authenticated via JWT subprotocol ('movr.jwt'), channel group event broadcasting.
Database & Cache	PostgreSQL 15 (psycopg 3.2, dj-database-url), Redis 7	UUID primary keys, indexed status fields, relation prefetching, SQLite & InMemoryChannelLayer dev fallback.
Fintech & Storage	Monnify REST, Paystack, Supabase Storage, Resend / SMTP	Virtual dedicated bank accounts, signed avatar/KYC URLs with strict TTLs, 6-digit OTP delivery.

2. Data Model & Mobility Domain State Machine

The backend domain architecture uses a hub-and-spoke relationship centered around **TravelPlan**, **TravelMatch**, and request entities. This abstraction elegantly unifies passenger commuting and package logistics into a single execution pipeline.

Entity Model	Core Attributes & Types	Lifecycle States & Transitions
TravelPlan	UUID, created_by (FK), origin/dest coordinates & names, departure_time, vehicle_type, seats_available, price_per_seat, is_live	draft → published → in_progress → completed / cancelled
RideRequest	UUID, requester (FK), matched_plan (FK), origin/dest coordinates, scheduled_time, seats_requested, note	open → matched → picked_up → completed / cancelled

DeliveryRequest	UUID, requester (FK), matched_plan (FK), pickup/dropoff coords, package_description, weight_kg, insured_value	open → matched → in_transit → delivered / damaged / cancelled
TravelMatch	UUID, travel_plan (FK), ride_request (FK), delivery_request (FK), match_type, agreed_price	proposed → accepted → active → completed / rejected / cancelled
TrackingSession / Event	UUID, travel_plan (OneToOne), actor, event_type (location/status/note/sos), coordinates, payload	idle → live → paused → ended

3. Codebase Audit: Identified Vulnerabilities & Bottlenecks

A rigorous audit of the current implementation reveals critical concurrency, algorithmic, and scaling bottlenecks that must be resolved prior to production traffic:

CRITICAL VULNERABILITY: Race Condition in Wallet Balances

In apps/payments/models.py (lines 21-29), `Wallet.credit()` and `Wallet.debit()` perform in-memory arithmetic (`self.balance += amount`) followed by `self.save()`. Under concurrent requests (e.g. concurrent webhook deposit and withdrawal), this executes an unsynchronized read-modify-write cycle, resulting in silent balance corruption. Furthermore, `debit()` lacks validation against `available_balance`, allowing negative balances.

Component	Current Implementation Defect	Operational Impact & Failure Mode
Mover Discovery	Substring SQL matching on place strings (<code>origin_name__icontains</code>).	Fails completely on corridor trips (e.g., Ikeja→Lekki misses Maryland→VI). Ignores real coordinates and proximity entirely.
GPS Ingestion	Every WebSocket GPS ping triggers synchronous <code>TrackingEvent.objects.create()</code> to PostgreSQL.	At 500 active drivers sending pings every 3s = 166 SQL INSERTs/sec, exhausting database connection pools and disk I/O.
Pricing Engine	Static <code>price_per_seat</code> or 1% insured parcel value.	Zero real-time responsiveness to peak traffic, bad weather, or extreme demand-supply imbalances.
WebSocket Client	<code>MobilityRealtimeService</code> lacks ping/pong heartbeats and exponential backoff.	Silent disconnections on mobile cellular handovers (4G/WiFi), leaving riders with frozen tracking maps.
Frontend Structure	88 presentation directories; simultaneous use of both <code>sizer</code> and <code>flutter_screenutil</code> .	Code sprawl, redundant notifiers, increased binary size, and inconsistent font/layout scaling across devices.

4. Technical Deep-Dive & Advanced Engineering Strategies

4.1 Spatial Corridor Matching with Uber H3 Hexagonal Hierarchical Index

To replace naive string matching, we introduce a discrete global grid system using **Uber H3**. The Earth is partitioned into hierarchical hexagonal cells. Hexagons provide equal distance to all 6 adjacent neighbors, enabling ultra-fast spatial search and corridor intersection.

The H3 Corridor Matching Pipeline:

- 1. Route Quantization:** When a mover publishes a route, convert the polyline coordinates into an ordered sequence of H3 resolution 8 cells (~460m diameter) via `h3.grid_path_cells()` or polyline sampling.
- 2. Spatial Indexing:** Store the cell set in Redis: `SADD mover:cells:<plan_id> cell_1 cell_2 ... cell_n`.
- 3. Rider Request Lookup:** Convert rider pickup and dropoff coordinates into cells `H_pickup` and `H_dropoff`. Find eligible movers via set intersection in Redis: `SISMEMBER mover:cells:<plan_id> H_pickup`.
- 4. Directional & Detour Verification:** Ensure $\text{index}(H_pickup) < \text{index}(H_dropoff)$ and calculate additional detour time Δt . If $\Delta t \leq 10$ mins, surface the mover.

4.2 Real-Time Positioning, Redis GEO Ingestion & Map Matching

To protect PostgreSQL from GPS write storms while maintaining sub-second tracking latency, decouple the ingestion path:

Pipeline Stage	Technology & Mechanism	Architectural Advantage
1. Ingestion	Daphne WebSocket → Redis GEOADD movers:live Ing lat mover_id	In-memory write executes in < 0.5ms with zero database contention.
2. Proximity	Redis GEOSEARCH movers:live FROMLONLAT Ing lat BYRADIUS 3 km	Sub-millisecond discovery of active drivers near any rider's location.
3. Broadcast	Django Channels Redis Channel Layer (group_send)	Pushes live coordinates directly to matched passenger/sender WebSocket rooms.
4. Persistence	Celery Worker / Redis Streams downsampled flush (every 30s)	Reduces SQL INSERT load by 90% while preserving audit logs and path history.
5. Client Smoothing	Flutter 1D/2D Kalman Filter + Tween Animation (LERP)	Filters out GPS multipath jitter and renders silky 60fps car marker movement.

4.3 Dynamic Surge Pricing Engine via Spatial Demand-Supply Ratios

A scalable surge pricing engine calculates dynamic fare multipliers based on localized demand and supply density within H3 hexagons (Resolution 7 ~ 1.2km):

Surge Algorithm Formulation:

- **Demand Counter:** `INCR demand:h3:<hex_id>` with a rolling 10-minute TTL.
- **Supply Counter:** `SCARD supply:h3:<hex_id>` reflecting active live movers in the hex.
- **Raw Ratio:** $R = \text{Demand} / \max(1, \text{Supply})$
- **Spatial Smoothing (K-Ring Convolution):** To avoid sudden pricing boundaries across adjacent streets, compute weighted average surge across neighbor cells: $S_smooth = 0.6 \times S_cell + 0.4 \times \text{avg}(S_k1_neighbors)$.
- **Fare Multiplier:** $S = \text{clamp}(1.0 + \alpha \times (R - \text{Threshold}), 1.0, 3.0)$.
- **Final Price:** $\text{Price} = [\text{BaseFare} + (\text{Distance} \times \text{Rate_km}) + (\text{Duration} \times \text{Rate_min})] \times S$

4.4 Spatial Telemetry, Driver Safety & Trip Analytics Pipeline

Moving beyond basic database counters, telemetry transforms GPS streams into actionable intelligence:

Telemetry Signal	Mathematical / Detection Model	Platform Action & Value
Route Adherence	Hausdorff Distance / Fréchet Distance between planned route polyline & actual GPS trace.	Automatic security alert when driver deviates > 500m from agreed corridor.
ETA Accuracy	Moving average error: $\Delta \text{ETA} = \text{ActualArrival} - \text{PredictedETA} $.	Continuously recalibrates machine learning traffic weights across corridors.
Driving Profile	Mobile accelerometer jerk detection ($\Delta a / \Delta t$) and speed delta.	Driver safety score; discounts platform commission for safe drivers.

OD Heatmaps	Origin-Destination H3 matrix aggregation (H3_origin → H3_destination).	Heatmap visualization for operations; nudges offline drivers to high-demand zones.
-------------	--	--

5. Step-by-Step Engineering Execution Roadmap

Phase	Core Deliverables & Engineering Tasks	Target Outcomes
Phase 1 Critical Fixes	• Wrap Wallet credit/debit with select_for_update() and F() expressions. • Add available_balance guards against negative wallet states. • Add WebSocket heartbeat ping/pong & exponential backoff in Flutter. • Remove duplicate responsive libraries (standardize on flutter_screenutil).	Zero financial race conditions; resilient real-time WebSocket connection on mobile networks.
Phase 2 Spatial Engine	• Integrate h3-py into Django mobility app. • Index TravelPlan polylines and Ride/Delivery requests to H3 resolution 8 cells. • Replace string SQL matching with H3 corridor set intersections. • Implement Redis GEOADD / GEOSEARCH for active mover positioning.	True geographic corridor matching with instant proximity search across transit corridors.
Phase 3 Dynamic Pricing	• Implement Redis demand/supply counters per H3 hex. • Build spatial smoothing k-ring convolution engine. • Integrate surge multiplier into fare estimation and mover bidding APIs.	Automated revenue optimization and supply balance during peak rush hours.
Phase 4 Telemetry & Scale	• Build async Celery pipeline for downsampled GPS trace persistence. • Implement Flutter Kalman filter + LERP marker animation. • Implement route deviation alerts and OD demand matrix dashboard.	Enterprise scalability (10k+ concurrent drivers) with smooth 60fps tracking UI.

Summary Recommendation: Movr has a strong domain architecture and excellent security fundamentals. Executing the geospatial (H3/PostGIS), concurrency, and real-time decoupling upgrades detailed above will elevate the platform from a functional MVP to an enterprise-grade mobility and logistics network.